permissionlesstech / **bitchat-android**

Code | Issues 228 | Pull requests 149 | Actions | Projects | Security and quality

Insights

bitchat-android

Public

Notifications | Fork 2k | ☆

About

decentralized mesh chat

Readme

GPL-3.0 license

Activity | Custom properties

7.9k stars | **94** watching | **2.0k** forks

Report repository

155 Branches | **44** Tags | Go to file | Go to file | Code

callebtc Merge pull request #1020 from permissionlesstech/release/2.2.0 — 20501fd · 3 days ago

.agents/skills	Test README screenshot crop ha…	3 months ago
.github	Prepare reproducible Wear 0.1.0 r…	2 months ago
app	Prepare 2.2.0 release	3 days ago
docs	Merge remote-tracking branch 'ori…	4 days ago
fastlane/metadata/android/…	f-droid: add metadata folder (#134)	last year
gradle	Merge remote-tracking branch 'ori…	4 days ago
tools	Fix minified release scanner initiali…	4 days ago
wear	Prepare 2.2.0 release	3 days ago
.dockerignore	build: enable reproducible Android…	3 months ago
.gitignore	build: enable reproducible Android…	3 months ago
.java-version	build: enable reproducible Android…	3 months ago
AGENTS.md	Update repository privacy instructi…	3 months ago
	chore: change license from MIT to …	8 months ago

LICENSE.md		
PRIVACY_POLICY.md	Restore PRIVACY_POLICY.md	2 months ago
README.md	Clarify README screenshot descr…	3 months ago
build.gradle.kts	fix: restore Android Studio sync wit…	3 months ago
gradle.properties	Enhancement/apk sharing (#632)	3 months ago
gradlew	init	last year
gradlew.bat	init	last year
settings-gradle.lockfile	build: enable reproducible Android…	3 months ago
settings.gradle.kts	Merge remote-tracking branch 'ori…	3 months ago

README GPL-3.0 license



bitchat for Android

A decentralized peer-to-peer messaging app with dual transport architecture: local Bluetooth mesh networks for offline communication and internet-based Nostr protocol for global reach. No accounts, no phone numbers, no central servers.

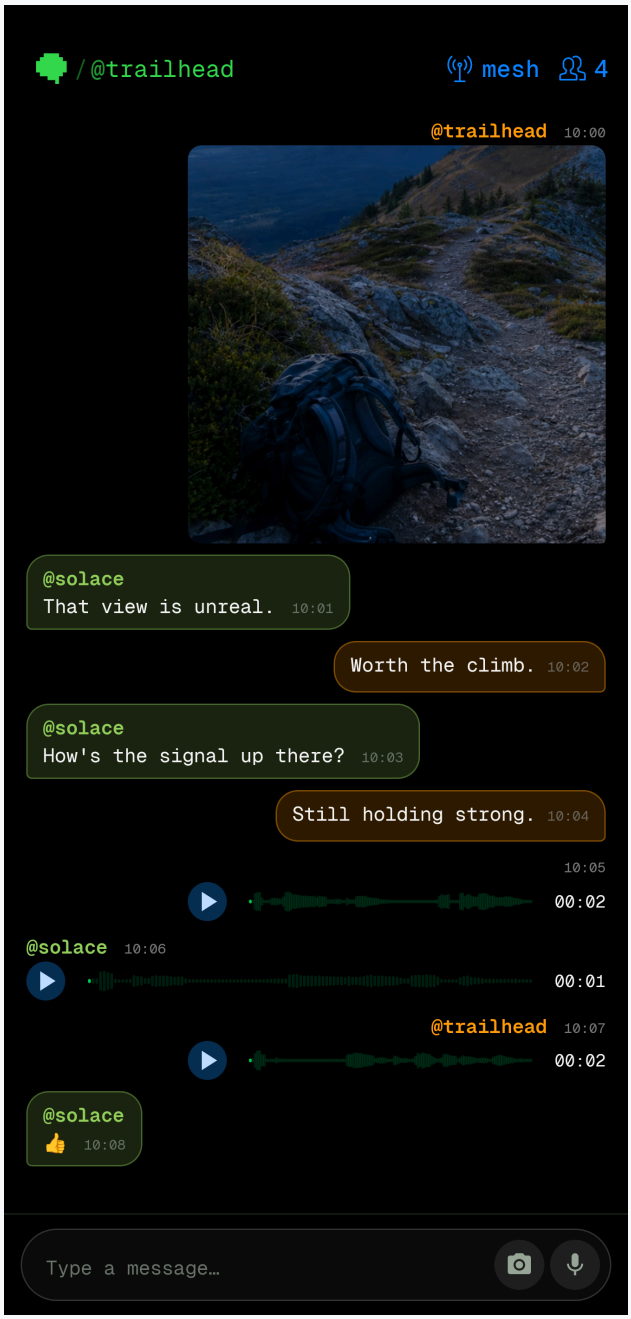
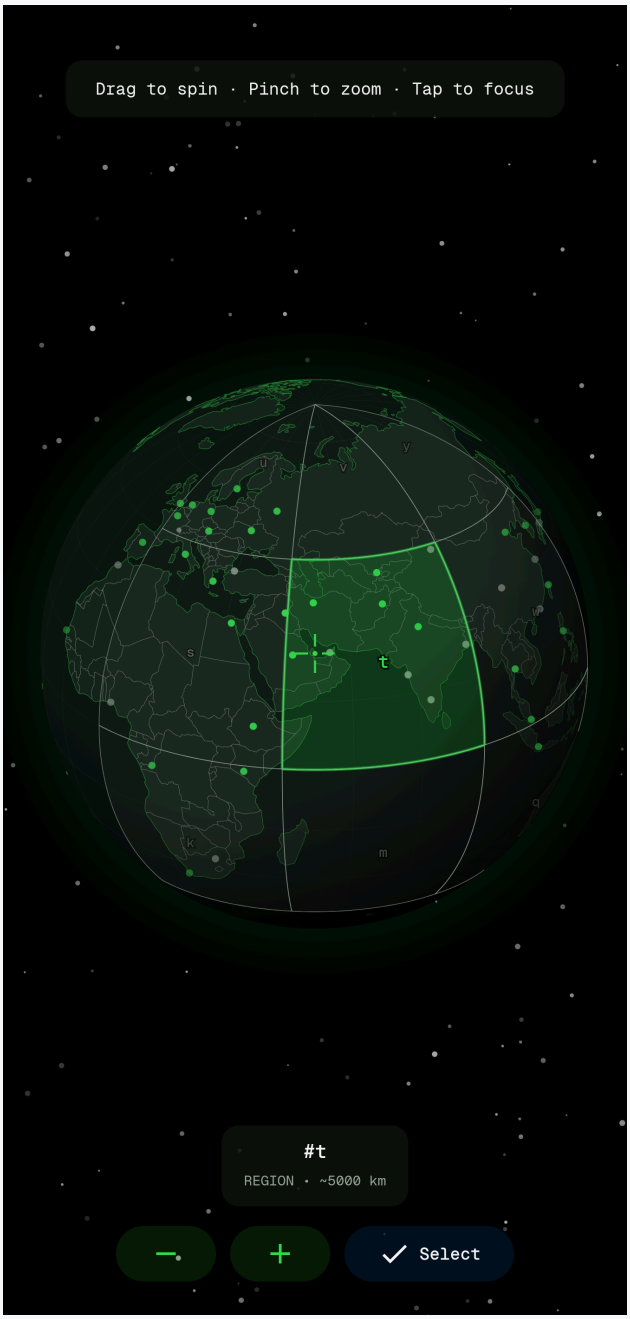
This is the Android implementation of bitchat, fully protocol-compatible with the iOS version for cross-platform mesh communication.

bitchat.free

GitHub Releases



See it in action

Offline mesh conversation	Geohash globe picker
	

License

This project is released into the public domain. See the LICENSE file for details.

Features

- **Dual Transport Architecture**: Bluetooth LE mesh for offline messaging, Nostr relays for internet-based messaging
- **Location-Based Channels**: Geographic chat rooms using geohash coordinates over Nostr relays

- **Intelligent Message Routing**: Automatically chooses the best transport, with queuing and retry when a peer is unreachable
- **End-to-End Encryption**: Noise Protocol (XX pattern, X25519 + ChaCha20-Poly1305) for private messages over the mesh
- **Decentralized Mesh Network**: Automatic peer discovery and multi-hop relay over Bluetooth LE (max 7 hops)
- **Wi-Fi Aware Transport**: Higher-bandwidth local mesh on supported devices
- **Channel Chats**: Topic-based group messaging with optional password protection (Argon2id + AES-256-GCM)
- **IRC-Style Commands**: Familiar `/join`, `/msg`, `/who` style interface
- **Tor Support**: Built-in Tor (Arti) for private internet connectivity
- **Emergency Wipe**: Triple-tap to instantly clear all data
- **Cross-Platform**: Binary protocol compatible with bitchat on iOS and macOS

Technical Architecture

Bluetooth Mesh Network (Offline)

- Direct peer-to-peer within Bluetooth range, multi-hop relay through nearby devices
- Noise Protocol sessions with forward secrecy; peer identities derived from static keys
- Compact binary packet format with fragmentation, TTL routing, and deduplication
- Adaptive duty cycling and connection limits for battery efficiency
- Foreground service keeps the mesh alive within Android background execution limits

Nostr Protocol (Internet)

- Global reach via public relays, geohash-based location channels
- Private messages fall back to Nostr for mutual favorites when the mesh is unavailable
- Ephemeral keys per geohash area

Android Stack

- Kotlin, Jetpack Compose (Material 3), MVVM
- Coroutines and Flow for all networking and state
- Core components: `MeshForegroundService` (persistent connectivity), `BluetoothMeshService` / `WifiAwareMeshService` (transports), `UnifiedMeshService` (transport selection), `NoiseSessionManager` (encryption sessions), `MessageRouter` (mesh/Nostr routing with outbox retry)

Building

Requires Android Studio and the Android SDK (API 26+).

```
git clone https://github.com/permissionlesstech/bitchat-android.git
cd bitchat-android
```

```
./gradlew assembleDebug
```

Install on a connected device:

```
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

The app requests Bluetooth, location (required for BLE scanning), and notification permissions at runtime.

Release APKs and the Android App Bundle can be rebuilt byte-for-byte in the pinned Linux container. Maintainers should follow the Android release guide. See Reproducible builds for the build trust model and public GitHub/Google Play verification procedures.

Testing

```
# Unit tests
./gradlew test

# Lint
./gradlew lint

# Instrumented tests (requires a device or emulator)
./gradlew connectedAndroidTest
```

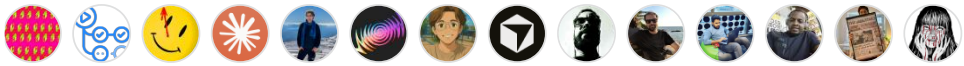
Note that BLE mesh behavior is difficult to emulate; protocol and session logic is covered by unit tests, while

Releases 39

Bitchat Android v2.2.0 Latest
3 days ago

+ 38 releases

Contributors 65



+ 51 contributors

Languages



Kotlin 87.1% **Java** 9.1% **Python** 2.3% **Shell** 1.1% **Other** 0.4%